

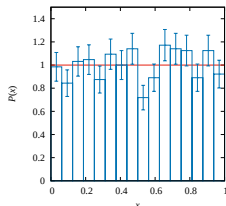
## 疑似乱数とは

- 計算機でプログラムに従って生成する乱数 (のようなもの)
- 乱数は何に役立つか？
  - ▶ 等式のチェック、例外の発見
  - ▶ 初期値にランダムネスを入れることで最悪の場合を避ける
  - ▶ サンプルングを使ったシミュレーション (→計算機実験 II)
- 乱数を使う場合の注意
  - ▶ 計算式に従って生成するため周期は有限であり、必ず何らかの相関がある
  - ▶ 初期化 (種の設定) を正しく行う
  - ▶ 実際にそれらしい乱数が生成されているか目で見て確認する
- 代表的な乱数発生器のひとつ: メルセンヌ・ツイスター
  - ▶ 周期  $2^{19937} - 1$ 、高速、日本製!
  - ▶ ヘッダファイル: [mersenne\\_twister.h](http://www.splnet.org/~scott/merenne_twister.h)
  - ▶ サンプルプログラム: [random.c](http://www.splnet.org/~scott/random.c)



# ヒストグラムの作り方

- 連続変数 (実数) のデータの場合 ([ ] 内はサンプルプログラムでの変数名)
  - ▶  $N$ : サンプル数 [samples]
  - ▶  $x_{\min}$ : データの最小値 (カットオフ) [xmin]
  - ▶  $x_{\max}$ : データの最大値 (カットオフ) [xmax]
  - ▶  $n$ : ビンの个数 [bins]
  - ▶  $\Delta$ : ビンの幅 ( $\Delta = (x_{\max} - x_{\min})/n$ ) [dx]
- サイズ  $n$  の配列を準備
  - ▶ データ毎にどのビンに入るか計算:  $j = (x - x_{\min})/\Delta$
  - ▶ (必要に応じて)  $0 \leq j < n$  であることを確認 (範囲外のデータは無視する)
  - ▶ 配列の  $j$  番目の値を 1 増やす
- サンプルプログラム: `histogram.c`  
(コンパイルには `mersenne_twister.h` と `cmatrix.h` が必要)



# ビンの個数の設定

- 最適の幅というものはない
- 個数を増やすと表現の自由度は増えるが、各ビンのエラーバーが大きくなる
  - ▶ データが統計的に独立である場合、それぞれのビンのカウント数  $m$  はポワソン分布に従う
  - ▶ 統計誤差  $\sim \sqrt{m}$
- いくつかの方法・公式が提案されているが、分布の形によっては不適切な場合も
  - ▶ スタージェスの公式  $n = \log_2 N + 1$
  - ▶ スコットの公式  $\Delta = 3.5\sigma/N^{1/3}$
- 実際には、ビンの個数を何通りか試してみるのが良い
- データを取り直すことが出来ない and/or コストがかかる場合も多いので、生データはいったんファイルに保存しておく